

TIME COMPLEXITY

HPC Practicals — OpenMP

Practical 1: BFS & DFS · Practical 2: Sorting · Practical 3: Reduction

Sequential vs Parallel · Best / Average / Worst Case · Space Complexity

With derivations, examples, and oral exam answers

$O(V+E)$

$O(n^2)$

$O(n \log n)$

$O(n/P)$

Practical 1 — BFS and DFS Time Complexity

Graph traversal algorithms visit every node and edge in the graph. Time complexity depends on the number of **Vertices (V)** and **Edges (E)**.

■ Notation: V = number of vertices (nodes) | E = number of edges | P = number of parallel processors/threads

BFS (Breadth-First Search)

Case	Sequential	Parallel	Space
Best Case	$O(V + E)$	$O(V + E/P)$	$O(V)$
Average Case	$O(V + E)$	$O(V + E/P)$	$O(V)$
Worst Case	$O(V + E)$	$O(V + E/P)$	$O(V)$

BFS Derivation:

- Each vertex is enqueued and dequeued exactly ONCE $\rightarrow O(V)$
- Each edge is examined exactly ONCE (when its source node is dequeued) $\rightarrow O(E)$
- Total sequential = $O(V) + O(E) = O(V + E)$
- Parallel: neighbor-checking loop parallelized with `#pragma omp parallel for` $\rightarrow E$ work split across P threads $\rightarrow O(E/P)$
- Parallel total = $O(V + E/P)$ [V is sequential — queue pop is single-threaded]

■ Real-world limitation: The `#pragma omp critical` section in BFS serializes all `queue.push()` operations. This creates a bottleneck — actual parallel speedup is less than the theoretical $O(E/P)$.

DFS (Depth-First Search)

Case	Sequential	Parallel	Space (call stack)
Best Case	$O(V + E)$	$O(V + E/P)$	$O(V)$ — balanced graph
Average Case	$O(V + E)$	$O(V + E/P)$	$O(V)$
Worst Case	$O(V + E)$	$O(V^2/P)$	$O(V)$ — linear graph (chain)

DFS Derivation:

- Visits every vertex once and every edge once $\rightarrow O(V + E)$ sequential
- Parallel: uses `#pragma omp task` — each unvisited neighbor spawned as independent task
- Tasks run concurrently on P threads $\rightarrow$ parallel time $\approx O(V + E/P)$

- Space complexity = $O(V)$ for the visited[] array + $O(V)$ for recursion call stack

Example calculation for the graph in code:

```
Graph: 6 vertices, 5 edges (undirected -> 10 directed edge checks)

Sequential BFS:  $O(6 + 10) = O(16)$  operations
Parallel BFS (2 threads):  $O(6 + 10/2) = O(11)$  operations [theoretical]

Sequential DFS:  $O(6 + 10) = O(16)$  operations
Parallel DFS (2 threads):  $O(6 + 10/2) = O(11)$  operations [theoretical]

Speedup =  $16/11 = \sim 1.45\times$  (limited by sequential queue operations)
```

Practical 2 — Bubble Sort and Merge Sort Time Complexity

Sorting algorithms arrange n elements in order. Time complexity depends on the number of comparisons and swaps. Parallel versions divide work among P threads.

■ Notation: n = number of elements | P = number of threads | $\log n$ = base-2 logarithm

Bubble Sort (Sequential)

Case	Time Complexity	Why	Space
Best Case	$O(n^2)$	No early termination in this implementation	$O(1)$
Average Case	$O(n^2)$	$\sim n^2/4$ comparisons on average	$O(1)$
Worst Case	$O(n^2)$	Reverse-sorted input — max swaps	$O(1)$

Bubble Sort Derivation:

- Outer loop runs $(n-1)$ times
- Inner loop runs $(n-i-1)$ times for pass i
- Total comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 = O(n^2)$

Parallel Bubble Sort — Odd-Even Transposition

Case	Sequential	Parallel	Space
Best Case	$O(n^2)$	$O(n^2/P)$	$O(1)$
Average Case	$O(n^2)$	$O(n^2/P)$	$O(1)$
Worst Case	$O(n^2)$	$O(n \times n/P)$	$O(1)$

Odd-Even Sort Derivation:

- Outer loop runs exactly n times (size iterations)
- Each phase (even or odd): inner loop does $n/2$ independent comparisons
- With P threads: each thread handles $(n/2)/P = n/(2P)$ comparisons per phase
- Total = n phases $\times$ $n/(2P)$ work = $n^2/(2P) = O(n^2/P)$
- Array is guaranteed sorted after n phases of Odd-Even Transposition

Example: $n=8$, $P=2$ threads

Sequential: $8 \times 4 = 32$ operations

Parallel: $8 \times (4/2) = 16$ operations per thread

Speedup = $32/16 = 2\times$ (ideal with 2 threads)

Merge Sort (Sequential)

Case	Time Complexity	Why	Space
Best Case	$O(n \log n)$	Always divides into 2 halves	$O(n)$
Average Case	$O(n \log n)$	$\log n$ levels, n work per level	$O(n)$
Worst Case	$O(n \log n)$	Always same regardless of input	$O(n)$

Merge Sort Derivation:

- Recurrence relation: $T(n) = 2T(n/2) + O(n)$
- By Master Theorem (case 2): $T(n) = O(n \log n)$
- There are $\log n$ levels of recursion (keep halving until size=1)
- At each level, total work for merging = $O(n)$
- Total = $\log n$ levels $\times O(n)$ per level = $O(n \log n)$

Parallel Merge Sort

Case	Sequential	Parallel	Space
Best Case	$O(n \log n)$	$O(n \log n / P)$	$O(n)$
Average Case	$O(n \log n)$	$O(n \log n / P)$	$O(n)$
Worst Case	$O(n \log n)$	$O(n \log n / P)$	$O(n)$

Parallel Merge Sort Derivation:

- Both halves are sorted independently as parallel tasks (#pragma omp task)
- Depth limited to 4 $\rightarrow$ creates up to $2^4 = 16$ parallel tasks at peak
- The parallel portion: sorting work = $O(n \log n)$, split across $P \rightarrow O(n \log n / P)$
- The merge step is SEQUENTIAL (cannot be easily parallelized) $\rightarrow O(n)$
- Dominant term: $O(n \log n / P)$ [for large n]

■ The merge step at the top level always takes $O(n)$ time sequentially. By Amdahl's Law, this limits maximum speedup to roughly $\log(n)$ times even with infinite processors.

Example: $n=1000$, $P=4$ threads

Sequential Merge Sort: $O(1000 \times \log_2(1000)) = O(1000 \times 10) = 10,000$ operations

Parallel Merge Sort: $O(10,000 / 4) = 2,500$ operations [theoretical]

Speedup = $10,000 / 2,500 = 4x$ (ideal with 4 threads)

Sequential Bubble Sort: $O(1000^2) = 1,000,000$ operations

Merge Sort is 100x FASTER than Bubble Sort for $n=1000$!

Practical 3 — Reduction Time Complexity (Min/Max/Sum/Avg)

Reduction operations scan the entire array once to compute an aggregate value. They are among the simplest and most efficiently parallelizable operations.

■ Notation: n = number of array elements | P = number of threads | $\log P$ = tree-reduction combination steps

All Four Operations

Operation	Sequential	Parallel (work)	Combination	Total Parallel	Space
Minimum	$O(n)$	$O(n/P)$	$O(\log P)$	$O(n/P + \log P)$	$O(1)$
Maximum	$O(n)$	$O(n/P)$	$O(\log P)$	$O(n/P + \log P)$	$O(1)$
Sum	$O(n)$	$O(n/P)$	$O(\log P)$	$O(n/P + \log P)$	$O(1)$
Average	$O(n)$	$O(n/P)$	$O(\log P)$	$O(n/P + \log P)$	$O(1)$

Reduction Derivation — Step by Step:

- Sequential: visit every element once $\rightarrow O(n)$ comparisons/additions, $O(1)$ space
- Parallel Phase 1 (work): each of P threads processes n/P elements $\rightarrow O(n/P)$ per thread
- Parallel Phase 2 (combine): P partial results are combined using a reduction tree
- Reduction tree has $\log P$ levels (like merge sort but for P values, not n)
- Total parallel = $O(n/P) + O(\log P)$
- For large n ($n \gg P$), $\log P$ is negligible $\rightarrow$ effectively $O(n/P)$

Reduction Tree visualization (P=4 threads):

```
Thread 0: sum[0..n/4-1] = partial_0 ■■
Thread 1: sum[n/4..n/2-1] = partial_1 ■■■ combine -> temp_01 ■■
Thread 2: sum[n/2..3n/4] = partial_2 ■■ ■■ FINAL SUM
Thread 3: sum[3n/4..n-1] = partial_3 ■■■ combine -> temp_23 ■■

Level 1: P/2 = 2 combinations (parallel)
Level 2: 1 final combination
Total combine steps =  $\log_2(4) = 2 \rightarrow O(\log P)$ 
```

Worked numerical example:

Array: [4, 7, 2, 9, 1, 6, 3, 8] n=8, P=4 threads

Sequential SUM: $4+7+2+9+1+6+3+8 = 40$ (8 additions) $O(8) = O(n)$

Parallel SUM:

Thread 0 -> [4, 7] -> partial = 11 (2 additions)

Thread 1 -> [2, 9] -> partial = 11 (2 additions)

Thread 2 -> [1, 6] -> partial = 7 (2 additions)

Thread 3 -> [3, 8] -> partial = 11 (2 additions)

Combine: $11+11 = 22$, $7+11 = 18$ (2 additions)

Final: $22+18 = 40$ (1 addition)

Total parallel additions = $2+2+2+2+1 = 11$

vs Sequential = 7 additions

Parallel wall-time = $O(n/P) = O(8/4) = O(2)$ <- each thread does 2 ops

Speedup = $O(n) / O(n/P) = P = 4\times$ (ideal)

Special cases — initial values matter!

Operation	Initial Value	Why this value?	Wrong init effect
Minimum	INT_MAX (~2.1 billion)	Any array element will be smaller, replacing it	Could miss actual minimum
Maximum	INT_MIN (~-2.1 billion)	Any array element will be larger, replacing it	Could miss actual maximum
Sum	0	Adding 0 doesn't change result (identity for +)	Wrong sum if non-zero init
Product	1	Multiplying by 1 doesn't change result	Wrong product if init=0

✓ OpenMP's reduction clause handles the initial value automatically based on the operator. Each thread's private copy is initialized to the identity value of the operation before the loop starts.

Master Summary — All Time Complexities

Practical	Algorithm	Best Case	Avg Case	Worst Case	Parallel	Space
P1	BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V+E/P)$	$O(V)$
P1	DFS	$O(V+E)$	$O(V+E)$	$O(V+E)$	$O(V+E/P)$	$O(V)$
P2	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2/P)$	$O(1)$
P2	Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n / P)$	$O(n)$
P3	Min / Max	$O(n)$	$O(n)$	$O(n)$	$O(n/P)$	$O(1)$
P3	Sum / Average	$O(n)$	$O(n)$	$O(n)$	$O(n/P)$	$O(1)$

Key Formulas for Oral Exam

Speedup = Sequential Time / Parallel Time
 Ideal Speedup = P (number of processors)
 Efficiency = Speedup / P (ideal = 1.0 = 100%)

 Amdahl's Law = $1 / (S + P/N)$
 S = serial fraction, P = parallel fraction, N = processors
 If 10% is serial $\rightarrow$ max speedup = $1/0.1 = 10x$ forever

 Reduction $O(n) \rightarrow O(n/P + \log P) \sim O(n/P)$ for large n
 Bubble Sort $O(n^2) \rightarrow O(n^2/P)$
 Merge Sort $O(n \log n) \rightarrow O(n \log n / P)$
 Graph BFS/DFS $O(V+E) \rightarrow O(V + E/P)$

Complexity Growth Order (slowest to fastest growing):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

 Our algorithms ranked best to worst:
 Reduction (P3): $O(n)$ <- FASTEST
 BFS / DFS (P1): $O(V+E)$ <- linear in graph size
 Merge Sort (P2): $O(n \log n)$ <- fast sorting
 Bubble Sort (P2): $O(n^2)$ <- SLOWEST

 For $n=10,000$:
 $O(n) = 10,000$ operations
 $O(n \log n) = 130,000$ operations ($\log_2 10000 \sim 13$)
 $O(n^2) = 100,000,000$ operations <- $10,000x$ slower than $O(n)$!

Good luck in your oral exam! Remember: parallel time $\approx$ sequential / P (ideal), and real speedup is always less due to synchronization overhead and Amdahl's Law.